Problem 1 — Find the Promos

Prototype: github.com/Anshu2003-commits/Kuku_assignment · **Raw outputs:** `outputs/` in that repo · **Run:** `python run.py --input "<pack>" --out outputs/`

Why this problem

It is the only one of the three with a real input pack and an output you can check against a stated definition. The pain is concrete and recurring: an editor watches 50 episodes of every show by hand, and Kuku ships a show a week, so the cost grows with the catalogue while the editor's hours don't. It is also narrow enough to finish — find moments and boundaries, not cut video.

Assumptions (the brief was silent on all of these)

1. Screenplay timestamps match the rendered video 1:1.
2. 20–60s inclusive; the hook must land in the first 3s.
3. Clips cut on beat boundaries, never mid-line.
4. "Makes sense on its own" is judged on the clip alone, not the series.
5. Multi-segment promos allowed; a continuous stretch preferred.
6. Spoiler risk rises with episode number — a proxy, since there is no audience data.

How it works

```
.txt scripts → [parse] → [candidates] → [judge: LLM] → [rank] → md/csv/json
               code       code           AI            code
```

The one design rule: the model never emits a timestamp. The parser turns every scene, line and action into a *beat* with an id and exact times — 50 episodes, **3,727 beats, 0 parse warnings**. The LLM only selects beat ids and writes prose; code maps ids back. A hallucinated timestamp is impossible by construction. Validation rejects any id not in the candidate, or any illegal duration — it fired **8 times in 200**, each logged in `run_report.json`.

Kept as code	Given to the AI
Parsing, timestamps, beat ids	Is this actually a hook?
20–60s limits, boundary snapping	Does it cut on something unresolved?
Candidate generation and pruning	Would a stranger follow it?
Ranking, diversity, dedupe	The hook / cliffhanger / why lines

Heuristics cut 3,727 beats to **200 candidates**, 4 per episode. That is cost control, not judgement: it fixes the per-show bill before the run starts. Ranking is code so a human can read and argue with the policy instead of finding it buried in a prompt. The model **tightened the boundaries on 118 of 200** candidates, so the beat-id mechanism is doing real work, not being rubber-stamped. The output is a ranked shortlist of 20 clips for an editor to approve — not a publishing decision.

Does it work?

Mechanically, checked by `eval/test_parse.py` and `eval/verify_outputs.py` (which re-parses the scripts independently and trusts nothing the pipeline says): **every emitted timestamp is a real beat boundary in the source**, every promo is 20–60s, 200/200 judged, 0 API failures.

Then I hand-labelled 30 clips — top of the slate, the tail, and the rejects — by reading each clip's **actual beats**, never the model's description of its own pick:

Slice	n	good	borderline	bad	
Top 10	10	6	3	1	**60% clean, 90% usable**
Whole slate	18	8	8	2	**44% clean, 89% usable**
Rejects	12	0	4	8	**67% correctly dropped**

Where it picks badly. *Soft endings* are the top failure (6/30): the model scores cliffhanger 4/5 and the clip still cuts on a resolution — #3 ends on a flirt. Absolute 1–5 scoring is the cause; `standalone` has sd 0.66 and **not one clip in 200**

scored below 2. It is also *slightly too harsh at the gate* — 4 of 12 sampled rejects were postable, one ending on the word "Arsenic." *Spoiler risk is dead weight*: 198 of 200 scored 1 or 2. And it *reads prose, not frames*, so a striking image with no dialogue is systematically underrated — a ceiling on any text-only approach, not a tunable bug. Stitched promos ship flagged `stitch_unverified_continuity` because the tool cannot confirm two shots cut together cleanly.

Three defects found by labelling were fixed and re-measured (`eval/before_after.md`): top-10 went **30% → 60% clean**, at the cost of reject precision falling 77% → 67%.

Scale and cost

200 calls/show ≈ 230k tokens. Two free-tier limits bind: **15 requests/minute**, which sets the ~16 minute wall time, and **500 requests/day**, which caps you at **~2.5 shows per day** — I hit the daily wall during testing, and it is the limit that would actually bite a team shipping weekly. A paid tier removes both: under a minute, cents per show. Cost scales with `--per-episode`, not episode length, so a new show's budget is known before it runs, and `--reuse-judgements` re-ranks a past run for free — which is how the ranking was tuned without spending quota. Swapping the model is a flag (`--provider gemini|openai|groq|anthropic`), or `--no-llm` to run with no key.

AI tools used, and where they got it wrong

Built with **Claude (Claude Code)**; **Gemini 3.1 Flash Lite** judges inside the tool. Four real errors, all caught by tests or measurement:

1. **Claude used a retired model id** (`gemini-2.0-flash`, 404) — the API had to tell us. Fixed by listing models from the API instead of trusting memory.
2. **Claude's windowing assumed beats were time-ordered.** They are not: `CAMERA` and `DIALOGUE` are parallel tracks that step backwards up to 1.0s in 8 places. Caught only because a test asserted the invariant.
3. **My own lexicon scored a cosy scene as violence** — `crack(ing)` matched "**crackling** hearth fire" and `fires?` matched "hearth **fire**", ranking a bandaging scene 19th.
4. **Bigger model, same answer, 100× the time.** A reasoning-grade Gemini took ~245s per judgement vs ~2.5s for flash-lite, returning identical scores. Separately, one run hung **two hours on 2.2 seconds of CPU** — the SDK has no default HTTP timeout.

What I'd do next

Replace absolute 1–5 scoring with **pairwise comparison** — which is what both the compressed distributions and the soft-ending failure point to. Learn the ranking weights from real watch-through data instead of hand-setting them. And sample keyframes for the top candidates to close the text-only gap.